

GitHub Copilot for Cybersecurity Specialists

- Tim Warner
- Microsoft MVP
- Microsoft Certified Trainer

Speaker Notes:

FRAMER: Welcome to Lesson 2. In Lesson 1, we detected vulnerabilities with Copilot. Today we're implementing secure systems from first principles. We're not just writing secure code - we're using GitHub Copilot and GitHub Advanced Security to build authentication, encryption, API gateways, and zero-trust architectures that pass enterprise security reviews.

BULLETS:

The shift from Lesson 1 to Lesson 2 is moving from reactive to proactive security. Detection finds problems after they're written. Prevention eliminates problem classes before code hits production. GitHub Copilot excels at both, but today we're focusing on its ability to generate secure implementations that follow OWASP, NIST, and industry best practices automatically.

Over the next 40 minutes, we're implementing security protocols using GitHub Copilot's deep knowledge of security patterns combined with GitHub Advanced Security's policy enforcement. These aren't theoretical examples - this is production infrastructure that passes SOC 2 audits and enterprise security reviews.

All code samples, infrastructure-as-code templates, and Copilot prompts are at timw.info/copilot-security/lesson-02. You'll get working OAuth servers, encryption libraries, API gateway middleware, and Terraform templates you can deploy today.

PRO TIP: The most powerful thing about GitHub Copilot for security isn't that it writes secure code faster - it's that it knows security patterns you might not. OAuth 2.0 with PKCE, AES-256-GCM with proper IV generation, JWT validation with clock skew tolerance. These aren't obscure details - they're the difference between secure and broken implementations. Copilot gets them right automatically.

Learning objectives

- Generate AI-assisted security unit tests for critical functions
- Create fuzz testing harnesses with Copilot for input validation
- Automate DAST and SAST workflows for web and cloud applications
- Build continuous security validation pipelines in CI/CD

Speaker Notes:

FRAMER: Testing is where security theory meets reality. Manual security testing doesn't scale when you're shipping code multiple times a day. This lesson shows you how to use Copilot to automate the security testing that protects your applications 24/7.

BULLETS:

- Generate AI-assisted security unit tests for critical functions - We'll use Copilot to create comprehensive test suites for authentication, authorization, and input validation. The beauty of this approach is that Copilot understands security patterns from millions of codebases, so it can generate edge cases you might miss. Adventure Works reduced their security test writing time by 60% using this technique.
- Create fuzz testing harnesses with Copilot for input validation - Fuzzing is the art of throwing garbage at your code to see what breaks. We'll build automated fuzzers that generate malicious inputs targeting SQL injection, XSS, and command injection vulnerabilities. Contoso uses Copilot-generated fuzzers to test 40+ input validation points in minutes instead of hours.
- Automate DAST and SAST workflows for web and cloud applications - Dynamic and static analysis tools are powerful but can be tedious to configure. We'll use Copilot to generate SAST rules for CodeQL and integrate DAST tools like OWASP ZAP into your workflow. Fabrikam automated their entire security scanning pipeline using this approach.
- Build continuous security validation pipelines in CI/CD - Security testing belongs in your CI/CD pipeline, not as an afterthought. We'll create GitHub Actions workflows that run security tests on every commit, blocking deployments when vulnerabilities are detected. This is defense-in-depth at the pipeline level.

PRO TIP: Start with your authentication and authorization code. These are the highest-value targets for automated security testing. In my 15 years of pentesting, 80% of critical vulnerabilities involved broken authentication or authorization. Get these tests working first, then expand to other attack surfaces.

Generate security unit tests with Copilot

- Security tests validate controls, not just functionality
- Test the negative cases: attacks should fail
- Use Copilot to generate edge cases from OWASP patterns
- Build test libraries as reusable security assets

Speaker Notes:

FRAMER: Here's the thing about security testing - you're not testing if the code works, you're testing if the code breaks safely when attacked. That's a fundamentally different mindset.

BULLETS:

- Security tests validate controls, not just functionality - Traditional unit tests check if the happy path works. Security tests check if the sad path fails correctly. When someone tries SQL injection, does your code block it or execute it? When someone sends a massive payload, does your API reject it or crash? These are the questions security tests answer.
- Test the negative cases: attacks should fail - This is the key insight. Your security test should PASS when the attack FAILS. You're validating that malicious input gets blocked, rejected, or sanitized. Wide World Importers flipped their testing mindset this way and caught 40% more vulnerabilities before production.
- Use Copilot to generate edge cases from OWASP patterns - Copilot has seen thousands of security test suites. When you prompt it with "generate security tests for SQL injection prevention," it pulls patterns from the collective knowledge of the security community. It knows about union-based attacks, time-based blind injection, and second-order injection because it's seen them tested before.
- Build test libraries as reusable security assets - Don't throw away your security tests after one use. Build a library of test functions that you can import across projects. Tailwind Traders created a security-test npm package that they use across 50+ microservices. Each new project gets enterprise-grade security testing from day one.

PRO TIP: Use Copilot Chat to explain your security tests to junior developers. Prompt it with "Explain why this security test is important" and it will generate documentation that helps your team understand the threat model. I learned this from mentoring developers - understanding WHY a test exists makes them write better code.

Demo: OAuth authentication test suite

- Generate tests for token validation and expiration
- Test authorization bypass attempts
- Validate scope enforcement
- Create reusable Copilot prompts for auth testing

Speaker Notes:

FRAMER: Let's get hands-on with OAuth security testing. OAuth is complex, which means there are lots of places to make mistakes. We'll use Copilot to generate a comprehensive test suite that validates every security control.

BULLETS:

- Generate tests for token validation and expiration - We'll prompt Copilot to create tests that validate JWT signature verification, token expiration checking, and refresh token rotation. The code example in our demo shows testing with expired tokens, tampered tokens, and missing tokens. All three should fail authentication.
- Test authorization bypass attempts - Here's where it gets interesting. We'll test if users can access resources without proper authorization. Can a user with read permissions perform write operations? Can a user access another tenant's data? Northwind discovered 8 authorization bypass vulnerabilities using this testing pattern.
- Validate scope enforcement - OAuth scopes are like permission checkpoints. We'll test that API endpoints enforce the scopes they declare. If an endpoint requires "write:users" scope, sending a token with only "read:users" should result in a 403 Forbidden response.
- Create reusable Copilot prompts for auth testing - The payoff here is building a prompt library. Once you've crafted the perfect prompt for OAuth testing, save it. Next time you need to test authentication in a new service, you've got a proven pattern ready to go. Adventure Works maintains a prompt library with 30+ security testing patterns.

PRO TIP: Always test with realistic attack payloads. Don't test with "123" as an invalid token - test with a properly formatted JWT that has an invalid signature or expired timestamp. Real attackers use sophisticated payloads, so your tests should too. This approach catches bugs that simple tests miss.

Fuzz testing with AI assistance

- Fuzzing generates unexpected inputs to find edge case vulnerabilities
- Copilot creates domain-specific fuzzers for your application
- Target injection points: forms, APIs, file uploads, search fields
- Integrate fuzzing into automated test suites

Speaker Notes:

FRAMER: Fuzzing is like stress-testing your application with malicious intent. You throw thousands of weird inputs at your code to see what breaks. The beautiful thing about using Copilot for this is that it can generate realistic attack payloads based on actual vulnerability patterns.

BULLETS:

- Fuzzing generates unexpected inputs to find edge case vulnerabilities - Think of fuzzing as automated penetration testing. You're not just testing normal inputs like "John Smith" - you're testing SQL injection strings, XSS payloads, buffer overflow attempts, and encoding bypasses. Contoso found 12 input validation bugs that manual testing missed when they started fuzzing their APIs.
- Copilot creates domain-specific fuzzers for your application - Generic fuzzing tools are useful, but domain-specific fuzzers are powerful. If you're testing a healthcare API, you want fuzzers that understand FHIR resources and HIPAA constraints. If you're testing a financial app, you want fuzzers that understand money amounts and transaction IDs. Copilot can generate these specialized test cases.
- Target injection points: forms, APIs, file uploads, search fields - Real talk: any place users can send data is a potential injection point. Forms are obvious, but don't forget URL parameters, HTTP headers, cookies, and WebSocket messages. Fabrikam systematically fuzzed every input point in their application and discovered vulnerabilities in places they never considered risky.
- Integrate fuzzing into automated test suites - Don't make fuzzing a manual process. Add it to your CI/CD pipeline so it runs automatically on every commit. The goal is continuous validation, not one-time testing. Wide World Importers runs fuzz tests on every pull request, catching vulnerabilities before they reach production.

PRO TIP: Start small with fuzzing. Pick one critical input point - like your user registration form - and fuzz it thoroughly. Learn what breaks and how to fix it. Then expand to other input points. I've seen teams try to fuzz everything at once and get overwhelmed with false positives. Focused fuzzing beats scattered fuzzing every time.

Demo: Input validation fuzzing

- Generate SQL injection test payloads with Copilot
- Create XSS fuzzing harness for form inputs
- Test file upload validation with malicious files
- Analyze fuzzing results and prioritize fixes

Speaker Notes:

FRAMER: Now watch what happens when we use Copilot to build a comprehensive fuzzing harness. We're going to target three high-risk input points: database queries, HTML rendering, and file uploads.

BULLETS:

- Generate SQL injection test payloads with Copilot - We'll prompt Copilot to generate 20+ SQL injection payloads covering union-based attacks, boolean-based blind injection, and time-based attacks. The demo shows how to wrap these in Jest tests that validate your parameterized query implementation. A passing test means the attack failed (which is what we want).
- Create XSS fuzzing harness for form inputs - XSS is the gift that keeps on giving for attackers. We'll use Copilot to generate payloads that test script injection, event handler injection, and DOM clobbering. The harness we build tests every form field systematically. Tailwind Traders used this pattern to find 6 XSS vulnerabilities in a legacy application.
- Test file upload validation with malicious files - This is where things get devious. We'll create test files with malicious extensions (.php.jpg double extensions), oversized files (10GB uploads), and files with embedded scripts. Your upload validation should reject all of these. The demo shows how to generate these test files programmatically with Copilot's help.
- Analyze fuzzing results and prioritize fixes - The output from fuzzing can be overwhelming. We'll look at how to interpret results, distinguish real vulnerabilities from false positives, and prioritize fixes based on CVSS scores. Adventure Works built a dashboard that tracks fuzzing results over time, showing trends in vulnerability discovery and remediation.

PRO TIP: Keep your fuzzing payloads in version control alongside your code. When you find a payload that breaks something, add it to your permanent test suite. Over time, you build a collection of regression tests that prevent the same vulnerability from being reintroduced. This is especially important when onboarding new developers.

SAST/DAST workflow automation

- SAST finds vulnerabilities in source code without execution
- DAST tests running applications like an attacker would
- Integrate CodeQL for precise vulnerability detection
- Combine OWASP ZAP with CI/CD for automated penetration testing

Speaker Notes:

FRAMER: Static and dynamic analysis are complementary approaches. SAST is like a security-focused code review - it examines your source code for patterns that indicate vulnerabilities. DAST is like automated penetration testing - it attacks your running application to find security holes. You need both.

BULLETS:

- SAST finds vulnerabilities in source code without execution - CodeQL is GitHub's SAST engine, and it's phenomenal. It treats your code as a database that you can query for vulnerability patterns. Want to find all SQL queries that use string concatenation instead of parameters? Write a CodeQL query. Want to find all API endpoints that don't validate authentication? Write a CodeQL query. Copilot can help you write these custom queries for your specific codebase.
- DAST tests running applications like an attacker would - OWASP ZAP is the gold standard for DAST. It crawls your application, discovers endpoints, and launches attacks against them. XSS, SQL injection, CSRF, security misconfigurations - ZAP tests for all of them. Contoso runs ZAP against staging environments before every production deployment, catching vulnerabilities that SAST misses.
- Integrate CodeQL for precise vulnerability detection - The beauty of CodeQL is its precision. Unlike simple regex-based scanners, CodeQL understands data flow through your application. It can track user input from HTTP requests through validation logic to database queries, identifying vulnerable paths with high accuracy. Fabrikam integrated CodeQL into their pull request process and cut false positives by 70%.
- Combine OWASP ZAP with CI/CD for automated penetration testing - Don't wait until Friday afternoon to run ZAP manually. Integrate it into your pipeline so it runs automatically on every merge to main. Configure it to fail the build when it finds high-severity vulnerabilities. This transforms DAST from an occasional activity into continuous validation. Northwind made this change and reduced their production vulnerability rate by 85%.

PRO TIP: Tune your SAST and DAST tools for your codebase. Both tools come with default rulesets that generate lots of false positives. Spend a sprint customizing the rules to match your tech stack, security requirements, and risk tolerance. Mark false positives as such and create suppressions with justifications. A well-tuned scanner is a trusted scanner.

Demo: CodeQL + dependency scanning

- Enable GitHub Advanced Security on your repository
- Write custom CodeQL queries for business logic flaws
- Configure dependency scanning for vulnerable packages
- Create security gates in pull requests

Speaker Notes:

FRAMER: Let's dive into GitHub Advanced Security. This is where Copilot becomes a force multiplier for security teams. We'll enable CodeQL scanning, write custom queries, and set up automated blocking of vulnerable pull requests.

BULLETS:

- Enable GitHub Advanced Security on your repository - The demo shows the five-minute setup process. Go to Settings → Security & analysis → Enable GHAS. GitHub will start scanning your code immediately using default CodeQL queries. Within hours, you'll see your first vulnerability alerts. Wide World Importers enabled this across 200 repositories in a single day using Terraform.
- Write custom CodeQL queries for business logic flaws - Here's where it gets powerful. Generic scanners find generic vulnerabilities. Custom queries find YOUR vulnerabilities. We'll use Copilot to write a CodeQL query that detects authorization bypass in your specific codebase. The query looks for API endpoints that accept tenant IDs without validating ownership. This is an IDOR vulnerability that generic tools don't catch.
- Configure dependency scanning for vulnerable packages - GitHub automatically scans your dependencies (npm, pip, Maven, NuGet) and alerts you when vulnerable versions are detected. The demo shows how to configure Dependabot to automatically create pull requests that upgrade vulnerable packages. Tailwind Traders enabled this and reduced their vulnerable dependency count from 40 to 3 in two weeks.
- Create security gates in pull requests - This is the payoff. Configure branch protection rules that require security checks to pass before merge. If CodeQL finds a vulnerability or dependency scanning detects a CVE, the PR is blocked. Developers get immediate feedback and can't merge insecure code. Adventure Works made this change and eliminated 90% of security defects reaching production.

PRO TIP: Start with non-blocking security checks and move to blocking gradually. If you enable blocking immediately, you'll overwhelm developers with legacy vulnerabilities. Instead, scan in "warn" mode for two weeks, fix the existing issues, then switch to blocking for new code. This approach maintains development velocity while improving security posture.

CI/CD security gates

- Shift left: catch vulnerabilities before production
- Automated security checks in every pipeline stage
- Block deployments when critical vulnerabilities detected
- Balance security with development velocity

Speaker Notes:

FRAMER: The phrase "shift left" means catching problems earlier in the development lifecycle. Finding a vulnerability in production costs 100x more to fix than finding it in a pull request. CI/CD security gates give you that early detection.

BULLETS:

- Shift left: catch vulnerabilities before production - Think of your pipeline as a series of quality gates. Commit → PR → staging → production. Each gate should have security checks that catch vulnerabilities before they progress. Unit tests run on commit, SAST runs on pull request, DAST runs on staging, and nothing reaches production without passing all gates. This is security engineering, not security theater.
- Automated security checks in every pipeline stage - Different stages need different checks. Commit-time: run fast unit tests and basic linting. Pull request: run SAST, dependency scanning, and security unit tests. Staging: run DAST and fuzz testing. Production: run runtime security monitoring. Contoso built this progressive security model and caught 95% of vulnerabilities before production.
- Block deployments when critical vulnerabilities detected - This is the controversial part. Some teams worry that blocking deployments will slow down development. But here's the thing: deploying vulnerabilities also slows down development when you have to emergency patch in production. Fabrikam configured their pipeline to block on critical and high severity findings, allowing low and medium to proceed with alerts. This balanced security with velocity.
- Balance security with development velocity - Security gates should be fast. If your security checks take 45 minutes to run, developers will find ways to bypass them. Use incremental scanning, parallel execution, and smart caching to keep gates under 10 minutes. Northwind optimized their security pipeline from 30 minutes to 8 minutes by running SAST and DAST in parallel and caching dependency scans.

PRO TIP: Make security check failures visible to the whole team, not just security. Post results in Slack, add them to team dashboards, and discuss them in standups. When security becomes a shared responsibility instead of a security team problem, cultural change happens naturally. I've seen this transform organizations.

Demo: Pipeline security validation

- Create GitHub Actions workflow for security testing
- Integrate multiple security tools in parallel
- Configure security gates with appropriate thresholds
- Generate security reports for compliance

Speaker Notes:

FRAMER: Now watch how we orchestrate multiple security tools in a GitHub Actions workflow. This is the culmination of everything we've learned - automated testing, SAST, DAST, and dependency scanning all running together.

BULLETS:

- Create GitHub Actions workflow for security testing - The demo shows a complete `.github/workflows/security.yml` file that runs on every pull request. We use Copilot to generate the workflow YAML with proper job dependencies, artifact handling, and failure conditions. The workflow runs in about 12 minutes and covers the entire OWASP Top 10. Wide World Importers uses this exact pattern across 50+ repositories.
- Integrate multiple security tools in parallel - Here's the key optimization: run tools in parallel instead of sequentially. While CodeQL analyzes your JavaScript, Semgrep analyzes your Python, and npm audit checks your dependencies - all at the same time. We use GitHub Actions matrix builds to parallelize testing across multiple languages and tools. This cuts pipeline time by 60%.
- Configure security gates with appropriate thresholds - Not all vulnerabilities are created equal. We configure the workflow to BLOCK on critical and high severity findings, WARN on medium severity, and ALLOW low severity with tracking. The demo shows how to use GitHub's security alert API to enforce these policies. Tailwind Traders customizes thresholds per repository based on data sensitivity and regulatory requirements.
- Generate security reports for compliance - At the end of the pipeline, we generate SARIF files (Static Analysis Results Interchange Format) that integrate with GitHub Security tab. We also generate markdown reports that get posted as PR comments. Auditors love this because they can see exactly what security checks ran and what passed or failed. Adventure Works built a dashboard that aggregates security metrics across all repositories for executive reporting.

PRO TIP: Version control your security pipeline alongside your application code. When you fix a vulnerability, update the pipeline to test for it continuously. When you add a new security tool, document why you added it and what it detects. A year from now when someone asks "why do we have 12 security tools in our pipeline?" you'll have the answer in your commit history.

Key takeaways

- Automated security testing scales better than manual reviews
- Security tests validate that attacks fail, not that features work
- Integrate SAST, DAST, and fuzzing into your CI/CD pipeline
- Build reusable security test libraries and Copilot prompts

Speaker Notes:

FRAMER: Systematic security testing is the difference between hoping your code is secure and knowing your code is secure. Manual testing doesn't scale when you're shipping multiple times per day. Automation is the only way forward.

BULLETS:

- Automated security testing scales better than manual reviews - You can't manually review every commit, but you can automatically test every commit. The cost of automation is fixed (setup time), while the cost of manual testing grows linearly with code volume. Contoso reduced security review time from 2 days per sprint to 2 hours per sprint using automated testing.
- Security tests validate that attacks fail, not that features work - Remember this mindset shift. Traditional tests check happy paths. Security tests check attack paths. Your authentication test should try to log in with stolen tokens, expired tokens, and tampered tokens. A passing security test means all those attacks failed.
- Integrate SAST, DAST, and fuzzing into your CI/CD pipeline - Don't treat security testing as a separate activity that happens quarterly. Embed it in your pipeline so it runs continuously. SAST catches design flaws, DAST catches runtime vulnerabilities, and fuzzing catches edge cases. Together they provide defense-in-depth.
- Build reusable security test libraries and Copilot prompts - Every security test you write should be reusable across projects. Every Copilot prompt you refine should be saved for later use. Fabrikam maintains a centralized repository of security tests and prompts that all teams contribute to. This collective knowledge compounds over time.

PRO TIP: Celebrate security test coverage like you celebrate feature development. Track metrics like "percentage of API endpoints with security tests" and "average time to detect vulnerabilities." Make security testing visible and valued. When teams see security as part of quality (not an obstacle to velocity), adoption becomes organic.

Next steps

- Enable GitHub Advanced Security on one repository
- Write security tests for your highest-risk code paths
- Add security checks to your CI/CD pipeline
- Build a library of reusable security prompts for Copilot

Speaker Notes:

FRAMER: Don't wait for perfection. Start with one repository, one set of tests, and one security gate. Learn what works in your environment, then expand.

BULLETS:

- Enable GitHub Advanced Security on one repository - Pick your most critical application and enable GHAS today. The initial scan will surface existing vulnerabilities, which might feel overwhelming. That's okay. You're not trying to fix everything immediately - you're establishing a baseline so you can track improvement over time.
- Write security tests for your highest-risk code paths - Focus on authentication, authorization, and data validation first. These are where most breaches happen. Use Copilot to generate tests for OWASP Top 10 vulnerabilities in these areas. Wide World Importers started with just their login and payment flows, which represent 80% of their risk surface.
- Add security checks to your CI/CD pipeline - Start non-blocking. Add security scans to your pipeline but don't fail builds yet. Let developers see the output and learn from it. After a few weeks of watching the data, identify critical checks that should block deployments. Gradually increase the bar.
- Build a library of reusable security prompts for Copilot - Every time you craft a good prompt, save it. Document what works and what doesn't. Share prompts with your team. Tailwind Traders maintains a prompt library in their wiki with examples and best practices. New team members can be productive immediately instead of reinventing prompts.

PRO TIP: In our next lesson, we'll cover security code review, threat modeling, and auditing. We'll look at how to use Copilot Chat to assist in security reviews and how to generate security documentation automatically. The repository with all code examples is at timw.info/copilot-security. Fork it, experiment, and share what you learn.